Lab 1: The Transform Calculator

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

1 Overview

In this lab, you will work with homogeneous transformation matrices to move objects through 3D space. You'll build a "transform calculator" that can compose, invert, and visualize transformations, then animate a cube moving through a sequence of poses.

Duration: 90 min in-lab + take-home exercises

Prerequisites: Lab 0 completed, Python environment working

2 Learning Objectives

By the end of this lab, you will be able to:

1. Create and manipulate SE(3) homogeneous transforms
2. Compose multiple transforms correctly
3. Compute transform inverses
4. Visualize objects moving through 3D space
5. Verify hand calculations with Python

3 Pre-Lab

3.1 Review Questions

Answer these before lab:

1. What is the structure of a 4×4 homogeneous transformation matrix?
2. Given H_1^0 and H_2^1, how do you compute H_2^0?
3. What is the formula for H^{-1}? (Write it out)
4. Why can't we use H^T as the inverse?

3.2 Hand Calculation

Compute the following by hand (show your work):

Given:

$$H_1^0 = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1. What rotation does this represent?
2. What is the translation vector?
3. Compute H^{-1}

---

4 Part 1: SE3 Basics (30 min)

4.1 1.1 Creating Homogeneous Transforms

```
from spatialmath import SE3
import numpy as np

# Pure translation
T_trans = SE3.Tx(2)  # Translate 2 units along x
print("Translation along x by 2:")
print(T_trans)

# Pure rotation
T_rot = SE3.Rz(90, 'deg')  # Rotate 90° about z
print("\nRotation about z by 90°:")
print(T_rot)

# Combined: rotation + translation
T_combined = SE3.Rt(SE3.Rz(90, 'deg').R, [2, 1, 0])
print("\nRotation + Translation:")
print(T_combined)
```

4.2 1.2 Extracting Components

```
from spatialmath import SE3

T = SE3.Rz(45, 'deg') * SE3.Tx(2)

# Extract rotation matrix (3x3)
R = T.R
print("Rotation matrix R:")
print(R)

# Extract translation vector (3x1)
```

```
t = T.t
print(f"\nTranslation vector t: {t}")

# Extract full 4x4 matrix
H = T.A
print("\nFull homogeneous matrix H:")
print(H)
```

4.3 1.3 Exercise 1: Build a Transform

Task: Create the following transform in Python:

- Rotate 90° about z-axis
- Then translate by [2, 1, 0]

Verify that your result matches:

$$H = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
from spatialmath import SE3
import numpy as np

# YOUR CODE HERE
T = ...

# Verify
expected = np.array([
    [0, -1, 0, 2],
    [1, 0, 0, 1],
    [0, 0, 1, 0],
    [0, 0, 0, 1]
])

print("Your result:")
print(T.A)
print(f"\nMatches expected? {np.allclose(T.A, expected)}")
```

---

5 Part 2: Transform Composition (30 min)

5.1 2.1 Composing Transforms

```
from spatialmath import SE3

# Define two transforms
T1 = SE3.Tx(1)           # Translate x by 1
T2 = SE3.Rz(90, 'deg')   # Rotate z by 90°

# Compose: T1 then T2
T_total = T1 * T2
print("T1 * T2 =")
print(T_total)

# Compare with reversed order
T_total_rev = T2 * T1
print("\nT2 * T1 =")
print(T_total_rev)

print(f"\nAre they equal? {np.allclose(T_total.A, T_total_rev.A)}")
```

5.2 2.2 Frame Chains

```
from spatialmath import SE3

# Frame chain: World -> Frame1 -> Frame2 -> Frame3
T_01 = SE3.Tz(1)                    # Frame 1 is 1m above world
T_12 = SE3.Rz(45, 'deg') * SE3.Tx(0.5)  # Frame 2 rotated and translated from 1
T_23 = SE3.Rx(90, 'deg')            # Frame 3 rotated from 2

# Compute frame 3 w.r.t. world
T_03 = T_01 * T_12 * T_23
print("Frame 3 in World coordinates:")
print(T_03)
print(f"\nPosition of Frame 3 origin: {T_03.t}")
```

5.3 2.3 Exercise 2: Robot Arm Chain

Scenario: A simple 2-link robot arm:

- Link 1: Rotates about z by θ_1, then translates 1m along x
- Link 2: Rotates about z by θ_2, then translates 0.5m along x

Task: Write a function to compute the end-effector transform given joint angles.

```
from spatialmath import SE3
import numpy as np
```

```python
def forward_kinematics(theta1_deg, theta2_deg):
    """
    Compute end-effector transform for 2-link arm.

    Args:
        theta1_deg: Joint 1 angle (degrees)
        theta2_deg: Joint 2 angle (degrees)

    Returns:
        SE3 transform of end-effector w.r.t. base
    """
    # YOUR CODE HERE
    T_01 = ...  # Base to Link 1
    T_12 = ...  # Link 1 to Link 2 (end-effector)

    T_02 = T_01 * T_12
    return T_02

# Test cases
print("theta1=0°, theta2=0°:")
T = forward_kinematics(0, 0)
print(f"  End-effector position: {T.t}")

print("\ntheta1=90°, theta2=0°:")
T = forward_kinematics(90, 0)
print(f"  End-effector position: {T.t}")

print("\ntheta1=45°, theta2=45°:")
T = forward_kinematics(45, 45)
print(f"  End-effector position: {T.t}")
```

---

6 Part 3: Transform Inverses (20 min)

6.1 3.1 Computing Inverses

```python
from spatialmath import SE3
import numpy as np

# Create a transform
T = SE3.Rz(90, 'deg') * SE3.Tx(2)
print("Original T:")
print(T)
```

```
# Compute inverse using spatialmath
T_inv = T.inv()
print("\nInverse T.inv():")
print(T_inv)

# Verify: T * T_inv = I
T_identity = T * T_inv
print("\nT * T_inv (should be identity):")
print(T_identity)
```

6.2 3.2 Manual Inverse Calculation

```
from spatialmath import SE3
import numpy as np

def manual_inverse(T):
    """
    Compute inverse of SE3 transform manually.

    H^{-1} = [R^T  | -R^T * d]
             [0    |    1    ]
    """
    R = T.R      # 3x3 rotation
    d = T.t      # 3x1 translation

    R_inv = R.T                  # R^T
    d_inv = -R.T @ d             # -R^T * d

    # Build 4x4 matrix
    H_inv = np.eye(4)
    H_inv[:3, :3] = R_inv
    H_inv[:3, 3] = d_inv

    return SE3(H_inv)

# Test
T = SE3.Rz(90, 'deg') * SE3.Tx(2)
T_inv_manual = manual_inverse(T)
T_inv_library = T.inv()

print("Manual inverse:")
print(T_inv_manual)
print("\nLibrary inverse:")
print(T_inv_library)
```

```
print(f"\nMatch? {np.allclose(T_inv_manual.A, T_inv_library.A)}")
```

6.3 3.3 Exercise 3: Verify Pre-Lab Calculation

Task: Verify your hand calculation of H^{-1} from the Pre-Lab.

```
from spatialmath import SE3
import numpy as np

# The original transform from Pre-Lab
H = np.array([
    [0, -1, 0, 2],
    [1, 0, 0, 1],
    [0, 0, 1, 0],
    [0, 0, 0, 1]
])
T = SE3(H)

# Compute inverse
T_inv = T.inv()

print("H^{-1} =")
print(T_inv.A)

# Enter your hand-calculated result here to verify
# your_answer = np.array([...])
# print(f"Matches hand calculation? {np.allclose(T_inv.A, your_answer)}")
```

---

7 Part 4: Orientation Parameterizations (20 min)

7.1 4.1 Euler Angles (Z-Y-Z)

The rotation matrix has 9 entries but only 3 degrees of freedom. Euler angles and Roll-Pitch-Yaw are compact 3-parameter representations.

```
from spatialmath import SO3, SE3
import numpy as np

# Create rotation from Z-Y-Z Euler angles ( ,  ,  )
R_euler = SO3.Eul(30, 45, 60, unit='deg')   # Rz(30°) * Ry(45°) * Rz(60°)
print("Z-Y-Z Euler (30°, 45°, 60°):")
print(R_euler)
```

```
# Extract Euler angles back
angles = R_euler.eul(unit='deg')
print(f"\nExtracted:  ={angles[0]:.1f}°,  ={angles[1]:.1f}°,  ={angles[2]:.1f}°")
```

7.2 4.2 Roll-Pitch-Yaw

```
from spatialmath import SO3
import numpy as np

# Create rotation from RPY (rotations about fixed world axes)
R_rpy = SO3.RPY(10, 20, 30, unit='deg')   # Rz(yaw) * Ry(pitch) * Rx(roll)
print("RPY (roll=10°, pitch=20°, yaw=30°):")
print(R_rpy)

# Extract RPY back
rpy = R_rpy.rpy(unit='deg')
print(f"\nExtracted: roll={rpy[0]:.1f}°, pitch={rpy[1]:.1f}°, yaw={rpy[2]:.1f}°")
```

7.3 4.3 Gimbal Lock Demonstration

When Euler angles reach a singularity, one degree of freedom is lost.

```
from spatialmath import SO3
import numpy as np

# Z-Y-Z gimbal lock:   = 0° aligns the first and third axes
R_lock = SO3.Rz(90, 'deg')   # This is Rz(90°) with  =0
print("R = Rz(90°):")
print(R_lock)

# Try to extract Z-Y-Z Euler angles
angles = R_lock.eul(unit='deg')
print(f"\nExtracted Euler:  ={angles[0]:.1f}°,  ={angles[1]:.1f}°,  ={angles[2]:.1f}°")
print("Only  +  is determined (gimbal lock)!")
print(f"  +   = {angles[0] + angles[2]:.1f}° (should be 90°)")

# RPY gimbal lock: pitch = ±90° aligns roll and yaw axes
R_rpy_lock = SO3.Ry(90, 'deg')
rpy = R_rpy_lock.rpy(unit='deg')
print(f"\nRPY of Ry(90°): roll={rpy[0]:.1f}°, pitch={rpy[1]:.1f}°, yaw={rpy[2]:.1f}°")
print("Only roll-yaw is determined (gimbal lock)!")
```

7.4 4.4 Exercise 4: Verify Euler Extraction

Task: Given the rotation matrix from the slides:

$$R = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

1. Extract Z-Y-Z Euler angles using `spatialmath`
2. Extract Roll-Pitch-Yaw angles
3. Verify that both reconstruct the same R

```
from spatialmath import SO3
import numpy as np

R = np.array([
    [0, 0, 1],
    [0, 1, 0],
    [-1, 0, 0]
])

# YOUR CODE HERE
# 1. Create SO3 from numpy matrix
# 2. Extract Euler and RPY angles
# 3. Reconstruct R from each and verify
```

---

8 Take-Home Exercises

The following exercises are to be completed **outside of lab**. They build on the in-lab work and focus on 3D visualization and animation using the transforms you practiced above.

9 Part 5: Visualizing Moving Objects (take-home)

9.1 5.1 Plotting Coordinate Frames

```
from spatialmath import SE3
import matplotlib.pyplot as plt

# Create transforms
T0 = SE3()                          # World frame (identity)
T1 = SE3.Tz(1) * SE3.Rx(30, 'deg')
T2 = T1 * SE3.Tx(1) * SE3.Rz(45, 'deg')
```

```
# Plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

T0.plot(ax=ax, frame='W', color='black', length=0.5)
T1.plot(ax=ax, frame='1', color='red', length=0.5)
T2.plot(ax=ax, frame='2', color='blue', length=0.5)

ax.set_xlim([-2, 2])
ax.set_ylim([-2, 2])
ax.set_zlim([0, 3])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Frame Chain Visualization')

plt.show()
```

9.2 5.2 Defining a Cube

```
import numpy as np

def create_cube(size=1.0):
    """
    Create vertices of a cube centered at origin.
    Returns 8x3 array of vertex coordinates.
    """
    s = size / 2
    vertices = np.array([
        [-s, -s, -s],
        [ s, -s, -s],
        [ s,  s, -s],
        [-s,  s, -s],
        [-s, -s,  s],
        [ s, -s,  s],
        [ s,  s,  s],
        [-s,  s,  s]
    ])
    return vertices

# Edges connect these vertex pairs
edges = [
    (0, 1), (1, 2), (2, 3), (3, 0),  # Bottom face
    (4, 5), (5, 6), (6, 7), (7, 4),  # Top face
```

```
    (0, 4), (1, 5), (2, 6), (3, 7)   # Vertical edges
]
```

9.3 5.3 Transforming the Cube

```
from spatialmath import SE3
import numpy as np
import matplotlib.pyplot as plt

def transform_cube(vertices, T):
    """Transform cube vertices by SE3 transform T."""
    transformed = []
    for v in vertices:
        # Transform each vertex (add 1 for homogeneous coords)
        v_transformed = T * v
        transformed.append(v_transformed)
    return np.array(transformed)

def plot_cube(ax, vertices, edges, color='blue', alpha=0.5):
    """Plot a cube given its vertices and edges."""
    for e in edges:
        pts = vertices[list(e)]
        ax.plot3D(pts[:, 0], pts[:, 1], pts[:, 2], color=color, alpha=alpha)

# Create cube and transform
cube = create_cube(0.5)
T = SE3.Tz(1) * SE3.Rz(45, 'deg')
cube_transformed = transform_cube(cube, T)

# Plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

plot_cube(ax, cube, edges, color='blue', alpha=0.3)
plot_cube(ax, cube_transformed, edges, color='red', alpha=0.8)

SE3().plot(ax=ax, frame='W', length=0.3)
T.plot(ax=ax, frame='T', length=0.3)

ax.set_xlim([-1.5, 1.5])
ax.set_ylim([-1.5, 1.5])
ax.set_zlim([-0.5, 2])
ax.set_title('Original Cube (blue) and Transformed Cube (red)')

plt.show()
```

9.4 5.4 Exercise 5: Animate the Cube

Task: Create an animation showing the cube moving through a sequence of transforms.

```
from spatialmath import SE3
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

def create_cube(size=0.5):
    s = size / 2
    return np.array([
        [-s, -s, -s], [ s, -s, -s], [ s,  s, -s], [-s,  s, -s],
        [-s, -s,  s], [ s, -s,  s], [ s,  s,  s], [-s,  s,  s]
    ])

edges = [(0,1),(1,2),(2,3),(3,0),(4,5),(5,6),(6,7),(7,4),(0,4),(1,5),(2,6),(3,7)]

def transform_cube(vertices, T):
    return np.array([T * v for v in vertices])

# Define motion sequence
def get_transform(t):
    """
    Returns transform at time t (0 to 1).
    Design a motion that combines rotation and translation.

    Hint: try composing SE3.Tz(), SE3.Rz(), SE3.Tx() with t-dependent values.
    """
    # YOUR CODE HERE
    pass

# Create animation
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

cube = create_cube(0.3)

def animate(frame):
    ax.clear()

    t = frame / 50  # 50 frames total
    T = get_transform(t)
```

```
    cube_t = transform_cube(cube, T)

    # Plot cube
    for e in edges:
        pts = cube_t[list(e)]
        ax.plot3D(pts[:, 0], pts[:, 1], pts[:, 2], 'b-', linewidth=2)

    # Plot world frame
    SE3().plot(ax=ax, frame='W', length=0.3)

    ax.set_xlim([-1, 3])
    ax.set_ylim([-2, 2])
    ax.set_zlim([-0.5, 3])
    ax.set_title(f'Frame {frame}/50, t={t:.2f}')

    return ax,

ani = FuncAnimation(fig, animate, frames=51, interval=100, blit=False)
plt.show()

# To save as video (requires ffmpeg):
# ani.save('cube_animation.mp4', writer='ffmpeg', fps=10)
```

9.5 5.5 Exercise 6: Pick-and-Place Motion

Task: Simulate a pick-and-place motion where the cube:

1. Starts at position A = [0, 0, 0]
2. Rises up to [0, 0, 1]
3. Moves to position B = [2, 0, 1]
4. Descends to [2, 0, 0]

Create a function that generates the transform at each stage.

```
def pick_and_place_transform(stage, progress):
    """
    Generate transform for pick-and-place motion.

    Args:
        stage: 0=rise, 1=move, 2=descend
        progress: 0 to 1 within the stage

    Returns:
        SE3 transform
    """
    # YOUR CODE HERE
```

```
    # stage 0: rise from z=0 to z=1
    # stage 1: move from x=0 to x=2 at z=1
    # stage 2: descend from z=1 to z=0 at x=2
    pass

# Test your function
for stage in range(3):
    for p in [0, 0.5, 1.0]:
        T = pick_and_place_transform(stage, p)
        print(f"Stage {stage}, progress {p}: position = {T.t}")
```

---

10 Appendix: Useful SE3 Methods

```
from spatialmath import SE3

T = SE3.Rz(45, 'deg') * SE3.Tx(1)

# Translation shortcuts
SE3.Tx(d)    # Translate along x
SE3.Ty(d)    # Translate along y
SE3.Tz(d)    # Translate along z

# Rotation shortcuts
SE3.Rx(angle, 'deg')  # Rotate about x
SE3.Ry(angle, 'deg')  # Rotate about y
SE3.Rz(angle, 'deg')  # Rotate about z

# Properties
T.R          # 3x3 rotation matrix
T.t          # 3x1 translation vector
T.A          # 4x4 homogeneous matrix

# Operations
T.inv()      # Inverse transform
T1 * T2      # Composition
T * point    # Transform a point

# Visualization
T.plot()     # Plot coordinate frame
```